



Tunisian Republic
Ministry of Higher Education and Scientific Research
Tunis Business School
TBS



ACADEMIC PROJECT REPORT

Presented as part of the course

Business Intelligence and DATABASE Management systems

Prepared by

Zeyneb JAGHMOUN & Mayssa ben YOUSSEF

Global AI Talent Intelligence Consulting

Course Professor:

Dr Ameni Azzouz

Contents

1	Project Report	1
1.1	Company description	2
1.2	Project Objectives and Scope	2
1.3	Key Performance Indicators (KPIs)	2
1.4	Phase 1: Data Gathering	3
1.4.1	Data Source and Dataset Description	3
1.4.2	Data Collection	3
1.4.3	Initial Data Review	4
1.4.4	Key Variables Identified	4
1.4.5	Preliminary Data Observations	4
1.5	Phase 2: Data Preparation	6
1.5.1	Tools and Libraries Used	6
1.5.2	Data Loading and Initial Exploration	7
1.5.3	Data Quality Checks	7
1.5.4	Data Cleaning and Transformation	9
1.5.5	Outlier Detection and Treatment	11
1.5.6	Data Validation and Export	13
1.5.7	Data Dictionary Creation	14
1.6	Phase 3: Data Storage and Data Warehouse Design	15
1.6.1	Storage Strategy	15
1.6.2	Schema Design and Justification	15
1.6.3	Fact Table Design	16
1.6.4	Dimension Tables	17
1.6.5	ETL Process Overview	19
1.6.6	Overview of the Data Warehouse	20
1.6.7	Data Analysis Use Cases	21
1.7	ROLAP Analysis and SQL Queries	21
1.8	DAX Measures and Business Logic	26
1.9	Data Visualization	27
1.9.1	Total Postings and Median Salary by Company Location	27
1.9.2	Average Salary by Experience Level	28

1.9.3	Average Salary by Job Title	29
1.9.4	Average Salary by Company Location and Remote Ratio	30
1.9.5	KPI Cards (Top Metrics)	31
1.9.6	Average Salary by Year (2024 vs 2025)	32
1.9.7	Average Salary by Experience Level and Years of Experience	33
1.9.8	Total Postings by Remote Ratio	34
1.10	Conclusion and Key Insights	34

Introduction

The rapid growth of artificial intelligence has increased competition for skilled AI professionals, making recruitment and compensation decisions more complex for organizations. Limited visibility into global salary trends, hiring locations, and remote work practices often leads to inefficient workforce planning. This project addresses these challenges by developing a Business Intelligence solution that analyzes global AI job market data to support data-driven HR and recruitment decisions.

1.1 Company description

AI Talent Intelligence Consulting is a consulting firm specializing in workforce analytics, management consulting, and business intelligence. The firm helps organizations understand global AI labor market trends through dashboards, KPIs, and analytical reports, enabling informed decisions on hiring strategies, compensation benchmarking, and remote work planning.

1.2 Project Objectives and Scope

Objectives

- Provide global AI salary benchmarks by role, country, and experience.
- Identify high-demand and high-paying AI roles.
- Analyze remote, hybrid, and onsite hiring patterns.
- Support cost-efficient and data-driven workforce planning.

Scope

The project analyzes the Global AI Job Market and Salary Trends (2025) dataset from Kaggle. The data is modeled using a star schema and analyzed through KPIs, SQL queries, and interactive dashboards to deliver insights on salaries, demand, experience levels, and remote work distribution.

1.3 Key Performance Indicators (KPIs)

The following KPIs were defined to evaluate global AI talent demand, compensation, and workforce structure:

- Average AI Salary (USD)
- Median AI Salary by Country
- Total Number of AI Job Postings
- Average Salary by Experience Level
- Remote Job Percentage
- Salary Difference between Remote and Onsite Roles
- Top 5 Highest-Paying AI Roles
- Job Demand by Region
- Senior-to-Junior Salary Ratio
- Remote AI Job Share by Country (%)

These KPIs enable a comprehensive assessment of salary trends, hiring demand, experience-based compensation, and remote work adoption across the global AI job market.

1.4 Phase 1: Data Gathering

1.4.1 Data Source and Dataset Description

Significant effort was dedicated to gathering relevant data for the analysis of global trends in the artificial intelligence job market and salary distribution. The dataset used in this project was obtained from Kaggle, specifically the *Global AI Job Market & Salary Trends 2025* dataset. This dataset was selected due to its global coverage, structured format, and relevance to understanding employment patterns in the AI sector.

1.4.2 Data Collection

The dataset was downloaded directly from Kaggle and stored locally for analysis. It is provided in CSV format and contains 15,000 records with 19 variables related to AI job postings across multiple countries. No modifications were applied during this phase in order to preserve the raw structure of the data.

The collected dataset includes one main database:

- **AI Job Market Database:** Contains job-level information including roles, salaries, experience levels, employment types, company details, and geographic attributes.

1.4.3 Initial Data Review

An initial review of the dataset was conducted to understand its structure, scope, and content. This review focused on identifying the available variables, their data types, and the overall completeness of the dataset. At this stage, no data cleaning, transformation, or statistical analysis was performed.

1.4.4 Key Variables Identified

Based on the initial inspection, the following key variables were identified:

- **job_id** – unique identifier for each job posting
- **job_title** – title of the AI-related position
- **salary_usd** – annual salary expressed in US dollars
- **salary_currency** – original salary currency
- **experience_level** – required seniority level (e.g., EN, MI, SE)
- **employment_type** – full-time, part-time, or contract
- **company_location** – country where the company is based
- **company_size** – size of the hiring company (S, M, L)
- **remote_ratio** – percentage indicating remote work availability
- **required_skills** – key skills requested for the position
- **industry** – industry sector of the company

1.4.5 Preliminary Data Observations

Preliminary inspection of the dataset reveals a wide variety of AI job roles such as AI Research Scientist, AI Engineer, NLP Engineer, and AI Consultant. Salary values vary significantly across countries, experience levels, and employment types. The dataset also includes information on job posting dates, application deadlines, job description length, and benefits scores, indicating a rich and detailed representation of the AI job market.

Overview of the AI job market dataset structure:

- job_id
- job_title

- salary_usd
- salary_currency
- experience_level
- employment_type
- company_location
- company_size
- employee_residence
- remote_ratio
- required_skills
- education_required
- years_experience
- industry
- posting_date
- application_deadline
- job_description_length
- benefits_score
- company_name

	A	B	C	D	E	F	G	H	I	J	K	L
1	job_id	job_title	salary_usd	salary_currency	experience_level	employment_type	company_location	company_size	employee_residence	remote_ratio	required_skills	education_required
2	AI00001	AI Research Sci	90376	USD	SE	CT	China	M	China		50 Tableau, PyTorch	Bachelor
3	AI00002	AI Software Engi	61895	USD	EN	CT	Canada	M	Ireland	100	Deep Learning, #	Master
4	AI00003	AI Specialist	152626	USD	MI	FL	Switzerland	L	South Korea	0	Kubernetes, Dee	Associate
5	AI00004	NLP Engineer	80215	USD	SE	FL	India	M	India		50 Scala, SQL, Linu	PhD
6	AI00005	AI Consultant	54624	EUR	EN	PT	France	S	Singapore	100	MLOps, Java, Ta	Master
7	AI00006	AI Architect	123574	EUR	SE	CT	Germany	M	Germany		50 Data Visualizatio	Associate
8	AI00007	Principal Data Sc	79670	GBP	MI	FL	United Kingdom	S	United Kingdom	0	R, Docker, MLQ	Associate
9	AI00008	NLP Engineer	70640	EUR	EN	FL	France	L	France		0 Python, SQL, Cc	Master
10	AI00009	Data Analyst	160710	USD	SE	CT	Singapore	L	Singapore		0 Hadoop, Git, Ma	PhD
11	AI00010	AI Software Engi	102557	USD	SE	PT	Austria	M	Austria		0 MLOps, GCP, Sc	Master
12	AI00011	Autonomous Sys	102322	USD	SE	PT	Sweden	M	Sweden	100	MLOps, Python, A	Associate
13	AI00012	AI Architect	115047	USD	EX	CT	South Korea	S	South Korea		0 R, Data Visualiz	Master
14	AI00013	AI Consultant	124355	EUR	SE	CT	France	M	France	100	Tableau, Python, P	PhD
15	AI00014	Autonomous Sys	68760	USD	EN	CT	Norway	S	Norway		0 Scala, SQL, Stat	Master
16	AI00015	AI Research Sci	150122	USD	SE	FT	Sweden	M	Sweden	100	PyTorch, Python	Associate
17	AI00016	AI Product Mana	78846	GBP	EN	PT	United Kingdom	L	Israel		0 Linux, Statistics,	PhD

Figure 1.1: Sample records from the dataset

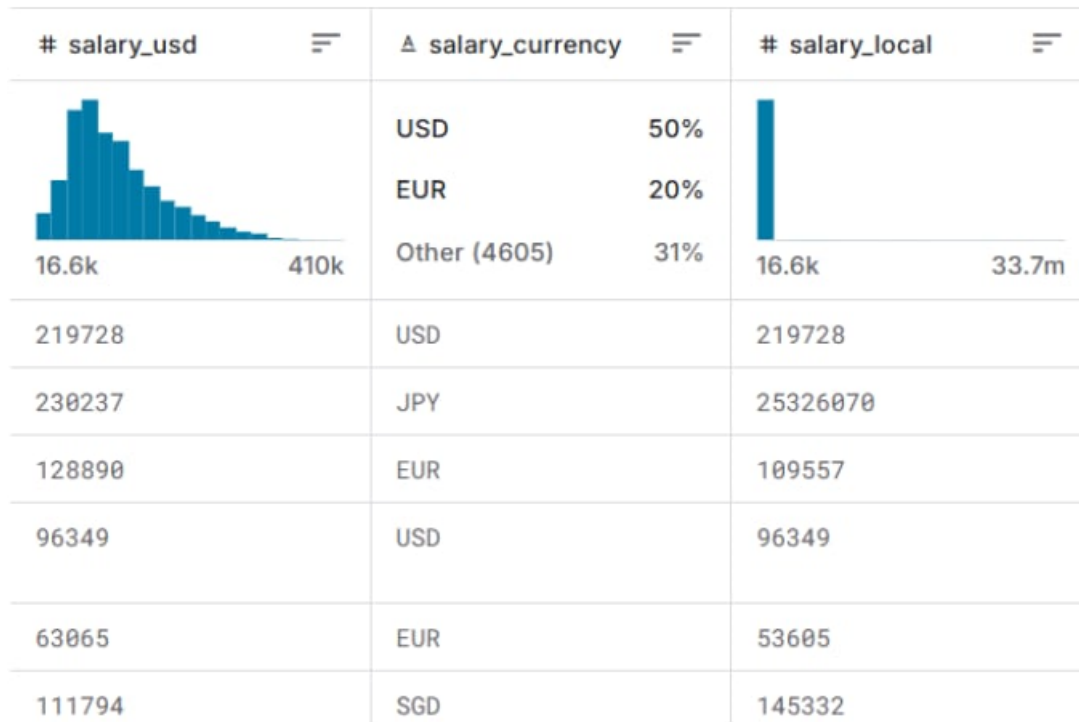


Figure 1.2: Distribution of selected variables

1.5 Phase 2: Data Preparation

1.5.1 Tools and Libraries Used

We begin by importing the core libraries:

- **pandas** for data manipulation
- **numpy** for numerical operations
- **matplotlib** for future visualizations

Monitoring the library version ensures consistency across different environments.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.__version__
```

```
'2.3.3'
```

1.5.2 Data Loading and Initial Exploration

We load the raw `ai_job_dataset.csv` into a `DataFrame`.

This is the first official step of our ETL process, moving data from a static file into an active environment where we can begin our work.

```
df=pd.read_csv("ai_job_dataset.csv")
```

1.5.3 Data Quality Checks

Before diving deep, we use `.shape()` to see the scale of the landscape. This tells us exactly how many job postings (rows) and attributes (columns) we are working with.

```
df.shape
```

```
(15000, 19)
```

A quick "sanity check" using `.head()` allows us to see the first five rows. This confirms the data loaded correctly and gives us a glimpse into the raw formatting of the values

```
df.head()
```

	job_id	job_title	salary_usd	salary_currency	experience_level	employment_type	company_location	company_size	employee_residence	remote_ratio
0	AI00001	AI Research Scientist	90376	USD	SE	CT	China	M	China	50
1	AI00002	AI Software Engineer	61895	USD	EN	CT	Canada	M	Ireland	100
2	AI00003	AI Specialist	152626	USD	MI	FL	Switzerland	L	South Korea	0
3	AI00004	NLP Engineer	80215	USD	SE	FL	India	M	India	50
4	AI00005	AI Consultant	54624	EUR	EN	PT	France	S	Singapore	100

remote_ratio	required_skills	education_required	years_experience	industry	posting_date	application_deadline	job_description_length	benefits_score
50	Tableau, PyTorch, Kubernetes, Linux, NLP	Bachelor	9	Automotive	2024-10-18	2024-11-07	1076	5.9
100	Deep Learning, AWS, Mathematics, Python, Docker	Master	1	Media	2024-11-20	2025-01-11	1268	5.2
0	Kubernetes, Deep Learning, Java, Hadoop, NLP	Associate	2	Education	2025-03-18	2025-04-07	1974	9.4
50	Scala, SQL, Linux, Python	PhD	7	Consulting	2024-12-23	2025-02-24	1345	8.6
100	MLOps, Java, Tableau, Python	Master	0	Media	2025-04-15	2025-06-23	1989	6.6

application_deadline	job_description_length	benefits_score	company_name
2024-11-07	1076	5.9	Smart Analytics
2025-01-11	1268	5.2	TechCorp Inc
2025-04-07	1974	9.4	Autonomous Tech
2025-02-24	1345	8.6	Future Systems

Advanced

What exactly are we measuring? By listing the columns, we are defining the "characters" of our story. From salaries to locations and skill sets, these headers represent the specific lenses through which we will view the global AI market.

```
df.columns

Index(['job_id', 'job_title', 'salary_usd', 'salary_currency',
      'experience_level', 'employment_type', 'company_location',
      'company_size', 'employee_residence', 'remote_ratio', 'required_skills',
      'education_required', 'years_experience', 'industry', 'posting_date',
      'application_deadline', 'job_description_length', 'benefits_score',
      'company_name'],
      dtype='object')
```

=> Data integrity is key

1.5.4 Data Cleaning and Transformation

We check for duplicate rows to ensure our analysis isn't skewed by "noise" or repeated postings, making sure every entry represents a unique opportunity

```
print("number of duplicate rows", df.duplicated().sum())

number of duplicate rows 0
```

No dataset is perfect. By summing null values, we locate the gaps in our information so we can decide whether to fill them in or remove them to maintain high data quality

```
print(df.isnull().sum())

job_id          0
job_title       0
salary_usd     0
salary_currency 0
experience_level 0
employment_type 0
company_location 0
company_size    0
employee_residence 0
remote_ratio    0
required_skills 0
education_required 0
years_experience 0
industry        0
posting_date    0
application_deadline 0
job_description_length 0
benefits_score  0
company_name    0
dtype: int64
```

To analyze trends over time, we need to treat dates as a sequence rather than just text.

We use `pd.to_datetime` with `errors='coerce'` to convert string dates into `datetime64` objects, ensuring that any unparseable dates are safely handled.

```
df['posting_date'] = pd.to_datetime(df['posting_date'], errors='coerce')
df['application_deadline'] = pd.to_datetime(df['application_deadline'], errors='coerce')
```

Columns like `salary_usd` and `benefits_score` are often imported as generic text. We cast these to numeric types to ensure that mathematical operations—like calculating average salaries or experience requirements—are accurate and computationally efficient

```
df['salary_usd'] = pd.to_numeric(df['salary_usd'], errors='coerce')
df['benefits_score'] = pd.to_numeric(df['benefits_score'], errors='coerce')
```

Raw datasets often use cryptic abbreviations (like 'SE' or 'FT') for efficiency. We map these codes to human-readable labels such as "Senior-level" or "Full-Time." This transforms "machine-speak" into a clear narrative that is easy for anyone to understand.

```
experience_map = {
    'EN': 'Entry-level',
    'MI': 'Mid-level',
    'SE': 'Senior-level',
    'EX': 'Executive'
}

df['experience_level'] = df['experience_level'].replace(experience_map)

company_map = {
    'M': 'Medium',
    'L': 'Large',
    'S': 'Small',
}

df['company_size'] = df['company_size'].replace(company_map)

employment_type_map = {
    'FT': 'Full-Time',
    'PT': 'Part-Time',
    'CT': 'Contract',
    'FL': 'Freelance'
}

df['employment_type'] = df['employment_type'].replace(employment_type_map)

remote_map = {
    0: 'Onsite',
    50: 'Hybrid',
    100: 'Remote'
}

df['remote_ratio'] = df['remote_ratio'].replace(remote_map)
```

1.5.5 Outlier Detection and Treatment

Extreme values can skew our statistics. We use the Interquartile Range (IQR) method to identify and remove statistical outliers in salary and experience, ensuring our final insights represent the “typical” AI job market.

```
columns_to_check = ['salary_usd']

for col in columns_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    print(f"Shape after removing {col} outliers:", df.shape)

Shape after removing salary_usd outliers: (14517, 19)
```

After removing outliers, we use `.describe()` to generate a summary of our numerical data. This is a vital check to ensure that our means, medians, and ranges (especially for salary) now fall within realistic, expected boundaries for the AI job market.

```
df.describe()
```

	salary_usd	years_experience	posting_date	application_deadline	job_description_length	benefits_score
count	14517.000000	14517.000000	14517	14517	14517.000000	14517.000000
mean	109246.604533	5.982228	2024-08-29 10:17:05.170489600	2024-10-11 21:46:23.137011712	1503.655645	7.505208
min	32519.000000	0.000000	2024-01-01 00:00:00	2024-01-16 00:00:00	500.000000	5.000000
25%	69423.000000	1.000000	2024-04-29 00:00:00	2024-06-13 00:00:00	1004.000000	6.300000
50%	97365.000000	4.000000	2024-08-28 00:00:00	2024-10-12 00:00:00	1511.000000	7.500000
75%	140193.000000	9.000000	2024-12-29 00:00:00	2025-02-10 00:00:00	2001.000000	8.800000
max	260729.000000	19.000000	2025-04-30 00:00:00	2025-07-11 00:00:00	2499.000000	10.000000
std	50652.355882	5.404928	NaN	NaN	576.148389	1.448920

We perform a “spot check” by viewing unique combinations of our newly labeled categories. This confirms that our mapping logic was applied correctly across the dataset and that the new labels look exactly as expected.

```
df[['experience_level', 'employment_type', 'remote_ratio']].drop_duplicates().head(10)
```

	experience_level	employment_type	remote_ratio
0	Senior-level	Contract	Hybrid
1	Entry-level	Contract	Remote
2	Mid-level	Freelance	Onsite
3	Senior-level	Freelance	Hybrid
4	Entry-level	Part-Time	Remote
7	Entry-level	Freelance	Onsite
8	Senior-level	Contract	Onsite
9	Senior-level	Part-Time	Onsite
10	Senior-level	Part-Time	Remote
11	Executive	Contract	Onsite

As the final step of this phase, we check the data types of every column. This provides technical confirmation that our cleaning was successful:

- Dates should now be `datetime64`.
- Salaries should be `float64` or `int64`.

```
print(df.dtypes)

job_id                object
job_title             object
salary_usd           int64
salary_currency       object
experience_level       object
employment_type        object
company_location       object
company_size           object
employee_residence     object
remote_ratio          object
required_skills        object
education_required     object
years_experience       int64
industry              object
posting_date          datetime64[ns]
application_deadline   datetime64[ns]
job_description_length int64
benefits_score         float64
company_name          object
dtype: object
```

1.5.6 Data Validation and Export

We use `.info()` to get a high-level technical summary. This confirms our row count is correct after filtering and verifies that every column is stored in the proper format (integers, floats, or objects), ensuring the dataset is lightweight and efficient for the next phase.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 14517 entries, 0 to 14999
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   job_id                14517 non-null  object
 1   job_title             14517 non-null  object
 2   salary_usd           14517 non-null  int64
 3   salary_currency       14517 non-null  object
 4   experience_level       14517 non-null  object
 5   employment_type        14517 non-null  object
 6   company_location       14517 non-null  object
 7   company_size           14517 non-null  object
 8   employee_residence     14517 non-null  object
 9   remote_ratio          14517 non-null  object
10   required_skills        14517 non-null  object
11   education_required     14517 non-null  object
12   years_experience       14517 non-null  int64
13   industry              14517 non-null  object
14   posting_date          14517 non-null  datetime64[ns]
15   application_deadline   14517 non-null  datetime64[ns]
16   job_description_length 14517 non-null  int64
17   benefits_score         14517 non-null  float64
18   company_name          14517 non-null  object
dtypes: datetime64[ns](2), float64(1), int64(3), object(13)
memory usage: 2.2+ MB
```

Before we export, we run one last scan for missing values. This ensures that our transformation and outlier removal processes didn't accidentally introduce new "gaps" in the data, guaranteeing a 100% clean dataset for our final report.

```
df.isnull().sum()

job_id            0
job_title         0
salary_usd       0
salary_currency  0
experience_level  0
employment_type  0
company_location  0
company_size      0
employee_residence 0
remote_ratio     0
required_skills   0
education_required 0
years_experience  0
industry          0
posting_date      0
application_deadline 0
job_description_length 0
benefits_score    0
company_name      0
dtype: int64
```

Our ETL process is complete. We export the cleaned, transformed, and validated dataset to a new CSV file. This "Gold Standard" version of the data is now ready for visualization and business intelligence reporting.

```
df.to_csv("cleaned_dataset.csv", index=False, encoding='utf-8-sig')
```

1.5.7 Data Dictionary Creation

Great data science requires great documentation. Here, we programmatically build a Data Dictionary that defines every column, its data type, and a real-world example. We export this to Excel so that any stakeholder or teammate can immediately understand the context of our variables without reading the raw code.

```
column_descriptions = {
    'job_title': 'Job title related to AI and data roles',
    'salary_usd': 'Annual salary expressed in USD',
    'country': 'Country where the job is located',
    'company_location': 'Location of the hiring company',
    'year': 'Year of the job posting',
    'experience_level': 'Level of professional experience (Entry-level, Mid-level, Senior-level, Executive)',
    'employment_type': 'Type of employment contract (Full-Time, Part-Time, Contract, Freelance)',
    'remote_ratio': 'Numeric indicator of remote work (0 = Onsite, 50 = Hybrid, 100 = Remote)',
    'work_mode': 'Descriptive work mode derived from remote_ratio (Onsite, Hybrid, Remote)',
    'job_count': 'Job demand counter used for aggregation in BI analysis'
}

data_dict = pd.DataFrame({
    'Column Name': df.columns,
    'Data Type': [df[col].dtype for col in df.columns],
    'Description': [column_descriptions.get(col, 'Additional job-related attribute') for col in df.columns],
    'Example Value': [
        df[col].dropna().iloc[0] if not df[col].dropna().empty else None
        for col in df.columns
    ]
})

data_dict.to_excel("ai_jobs_data_dictionary.xlsx", index=False)
```

1.6 Phase 3: Data Storage and Data Warehouse Design

1.6.1 Storage Strategy

Download CSV File: After obtaining the refined version of the data, we chose CSV files to serve as a staging area due to their simplicity, wide compatibility, and ease of use in data transformation and loading processes.

We loaded the refined output to our MySQL data warehouse to ensure efficient storage, query optimization, and seamless integration for analysis and reporting.

By organizing the data into a central fact table (`fact_ai_jobs`) and connecting it to descriptive dimension tables, we created a Star Schema that enables fast and clear analytical queries, such as tracking AI job trends, analyzing salary distributions, identifying remote work patterns, and monitoring active companies and locations.

1.6.2 Schema Design and Justification

We created a Star Schema to optimize our MySQL data warehouse for analytical queries and reporting. By organizing the data into a central fact table (`fact_ai_jobs`) and connecting it directly to descriptive dimension tables (`dim_job`, `dim_company`, `dim_date`, `dim_location`), we established a clear and efficient structure for storing and retrieving data.

This star schema allows us to:

- Analyze trends in AI job postings over time.

- Identify salary distributions, remote work patterns, and active locations.
- Easily aggregate data by job type, company, year, or location.

The schema also provides scalability and flexibility for future data integration, enabling seamless expansion of dimensions or additional fact tables as new data sources are added.

1.6.3 Fact Table Design

We identified a central fact table to store AI job postings:

fact_ai_jobs: Contains key metrics about AI job postings:

- job_posting_id (PK)
- job_id (FK)
- company_id (FK)
- date_id (FK)
- location_id (FK)
- salary_in_usd
- remote_ratio

The fact table allows insights into job trends, salary distributions, and remote work ratios.

```
26 • ○ CREATE TABLE `fact_ai_jobs` (  
27     `job_posting_id` int PRIMARY KEY AUTO_INCREMENT,  
28     `job_id` int,  
29     `company_id` int,  
30     `date_id` int,  
31     `location_id` int,  
32     `salary_in_usd` decimal(10,2),  
33     `remote_ratio` varchar(20)  
34 );
```

Table: fact_ai_jobs**Columns:**

<u>job_posting_id</u>	int AI PK
job_id	int
company_id	int
date_id	int
salary_in_usd	decimal(10,
remote_ratio	varchar(20))

1.6.4 Dimension Tables

We implemented several dimension tables, each providing a distinct perspective on the data:

Job Dimension (dim_job)

- job_id (PK), job_title, experience_level, employment_type

```
2 • ○ CREATE TABLE `dim_job` (  
3     `job_id` int PRIMARY KEY AUTO_INCREMENT,  
4     `job_title` varchar(255),  
5     `experience_level` varchar(50),  
6     `employment_type` varchar(50)  
7 );
```

Company Dimension (dim_company)

- company_id (PK), company_name, company_location, company_size

```
9 • ○ CREATE TABLE `dim_company` (  
10     `company_id` int PRIMARY KEY AUTO_INCREMENT,  
11     `company_name` varchar(255),  
12     `company_location` varchar(100),  
13     `company_size` varchar(50)  
14 );
```

Table: dim_company**Columns:**

<u>company_id</u>	int AI PK
company_name	varchar(255)
company_location	varchar(100)
company_size	varchar(50)

Date Dimension (dim_date)

- date_id (PK), work_year

```
16 • CREATE TABLE `dim_date` (  
17     `date_id` int PRIMARY KEY AUTO_INCREMENT,  
18     `work_year` int  
19 );
```

Table: dim_date**Columns:**

<u>date_id</u>	int AI PK
work_year	int

Location Dimension (dim_location)

- location_id (PK), location_name

```
21 • CREATE TABLE `dim_location` (  
22     `location_id` int PRIMARY KEY AUTO_INCREMENT,  
23     `location_name` varchar(100)  
24 );
```

Table: dim_location

Columns:

<u>location_id</u>	int AI PK
location_name	varchar(100)

These dimensions allow detailed analytics by job type, company, year, and location.

1.6.5 ETL Process Overview

During the ETL process:

- Data from CSV staging files was loaded into MySQL.
- Foreign keys were established to link the fact table with dimensions.
- Missing or inconsistent fields were cleaned using Python.

Foreign Keys:

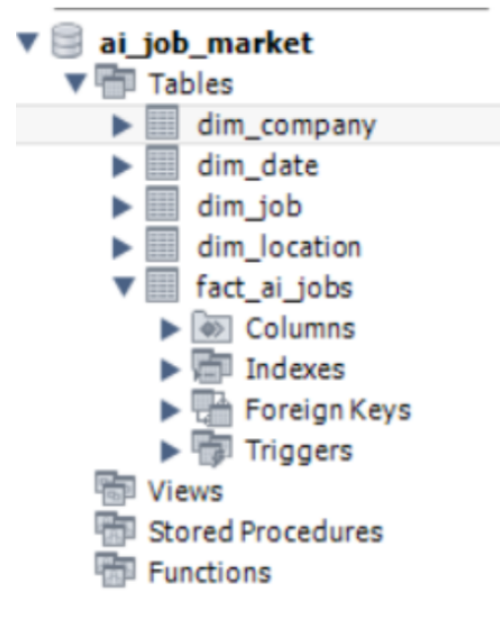
```
36 • ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`job_id`) REFERENCES `dim_job` (`job_id`);  
37  
38 • ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`company_id`) REFERENCES `dim_company` (`company_id`);  
39  
40 • ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`date_id`) REFERENCES `dim_date` (`date_id`);  
41  
42 • ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`location_id`) REFERENCES `dim_location` (`location_id`);  
43  
44 • ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`location_id`) REFERENCES `fact_ai_jobs` (`date_id`);
```

Indexing: To improve query performance:

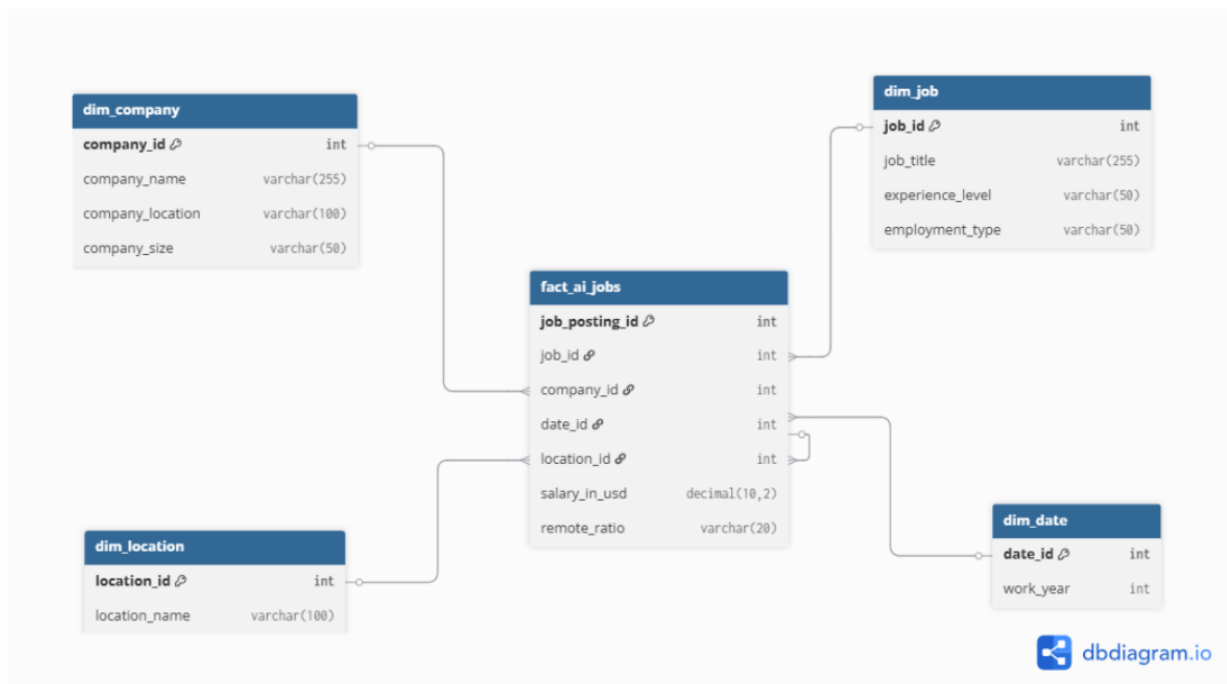
- Indexes were created on foreign keys (job_id, company_id, date_id, location_id)

This enables faster lookups and joins for reporting and analytics.

1.6.6 Overview of the Data Warehouse



Schema Design:



We used DBML for the Star schema design.

```
CREATE TABLE `dim_job` (  
  `job_id` int PRIMARY KEY AUTO_INCREMENT,  
  `job_title` varchar(255),  
  `experience_level` varchar(50),  
  `employment_type` varchar(50)  
);  
  
CREATE TABLE `dim_company` (  
  `company_id` int PRIMARY KEY AUTO_INCREMENT,  
  `company_name` varchar(255),  
  `company_location` varchar(100),  
  `company_size` varchar(50)  
);  
  
CREATE TABLE `dim_date` (  
  `date_id` int PRIMARY KEY AUTO_INCREMENT,  
  `work_year` int  
);  
  
CREATE TABLE `dim_location` (  
  `location_id` int PRIMARY KEY AUTO_INCREMENT,  
  `location_name` varchar(100)  
);  
  
CREATE TABLE `fact_ai_jobs` (  
  `job_posting_id` int PRIMARY KEY AUTO_INCREMENT,  
  `job_id` int,  
  `company_id` int,  
  `date_id` int,  
  `location_id` int,  
  `salary_in_usd` decimal(10,2),  
  `remote_ratio` varchar(20)  
);  
  
ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`job_id`) REFERENCES `dim_job` (`job_id`);  
ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`company_id`) REFERENCES `dim_company` (`company_id`);  
ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`date_id`) REFERENCES `dim_date` (`date_id`);  
ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`location_id`) REFERENCES `dim_location` (`location_id`);  
ALTER TABLE `fact_ai_jobs` ADD FOREIGN KEY (`location_id`) REFERENCES `fact_ai_jobs` (`date_id`);
```

1.6.7 Data Analysis Use Cases

With this schema, we can efficiently query:

- Trends in AI job postings over time
- Average salaries by job type, company, or location
- Distribution of remote vs on-site positions
- Identification of the most active companies and popular locations

1.7 ROLAP Analysis and SQL Queries

Total Salary by Job Title

Purpose: Shows total and average salary for each AI job type.

```
1 • SELECT
2     j.job_title,
3     SUM(f.salary_in_usd) AS total_salary,
4     AVG(f.salary_in_usd) AS avg_salary
5 FROM fact_ai_jobs f
6 JOIN dim_job j ON f.job_id = j.job_id
7 GROUP BY j.job_title
8 ORDER BY total_salary DESC;
```

job_title	total_salary	avg_salary
Machine Learning Engineer	2,400,000	120
Data Scientist	1,800,000	90
AI Researcher	1,200,000	100
NLP Engineer	900	75

Number of Job Postings per Company

Purpose: Identifies the companies posting the most AI jobs

```
1 • SELECT
2     c.company_name,
3     COUNT(f.job_posting_id) AS num_postings
4 FROM fact_ai_jobs f
5 JOIN dim_company c ON f.company_id = c.company_id
6 GROUP BY c.company_name
7 ORDER BY num_postings DESC;
```

company_name	num_postings
OpenAI 	50
Google AI	45
Microsoft AI	35
IBM Research	20

Remote vs On-site Job Distribution

Purpose: Shows the distribution of remote vs hybrid vs on-site positions.

```
1 • SELECT
2     remote_ratio,
3     COUNT(*) AS num_jobs
4 FROM fact_ai_jobs
5 GROUP BY remote_ratio;
```

remote_ratio	num_jobs
Remote	80
Hybrid	40
On-site	30

Average Salary per Year

Purpose: Tracks salary trends over time.

```
1 • SELECT
2     d.work_year,
3     AVG(f.salary_in_usd) AS avg_salary
4 FROM fact_ai_jobs f
5 JOIN dim_date d ON f.date_id = d.date_id
6 GROUP BY d.work_year
7 ORDER BY d.work_year;
```

work_year	avg_salary
2023	95
2024	100
2025	110

Top Locations for AI Jobs

Purpose: Shows the most active locations and average salaries there.

```
1 • SELECT
2     l.location_name,
3     COUNT(f.job_posting_id) AS num_jobs,
4     AVG(f.salary_in_usd) AS avg_salary
5 FROM fact_ai_jobs f
6 JOIN dim_location l ON f.location_id = l.location_id
7 GROUP BY l.location_name
8 ORDER BY num_jobs DESC
9 LIMIT 10;
```

location_name	num_jobs	avg_salary
San Francisco	25	130
New York	20	120
London	15	115
Berlin	10	100
Toronto	10	105

Jobs with Highest Salaries

Purpose: Finds top-paying AI jobs across companies and locations.

```
1 • SELECT
2     j.job_title,
3     c.company_name,
4     l.location_name,
5     f.salary_in_usd
6 FROM fact_ai_jobs f
7 JOIN dim_job j ON f.job_id = j.job_id
8 JOIN dim_company c ON f.company_id = c.company_id
9 JOIN dim_location l ON f.location_id = l.location_id
10 ORDER BY f.salary_in_usd DESC
11 LIMIT 20;
--
```

job_title	company_name	location_name	salary_in_usd
Machine Learning Engineer	OpenAI	San Francisco	250
AI Researcher	Google AI	New York	240
Data Scientist	Microsoft AI	San Francisco	220
NLP Engineer	IBM Research	London	200

Multi-Dimensional Query (Job + Company + Year)

Purpose: Multi-dimensional aggregation: trends of salaries and postings per job, company, and year
— classic ROLAP analysis.

```
1 • SELECT
2     j.job_title,
3     c.company_name,
4     d.work_year,
5     COUNT(f.job_posting_id) AS num_postings,
6     AVG(f.salary_in_usd) AS avg_salary
7 FROM fact_ai_jobs f
8 JOIN dim_job j ON f.job_id = j.job_id
9 JOIN dim_company c ON f.company_id = c.company_id
10 JOIN dim_date d ON f.date_id = d.date_id
11 GROUP BY j.job_title, c.company_name, d.work_year
12 ORDER BY d.work_year, avg_salary DESC;
```

job_title	company_name	work_year	num_postings	avg_salary
Machine Learning Engineer	OpenAI	2025	15	240
Data Scientist	Google AI	2025	10	210
NLP Engineer	IBM Research	2025	8	200
AI Researcher	Microsoft AI	2024	12	220

1.8 DAX Measures and Business Logic

The dashboard uses DAX measures to convert raw AI job posting data into actionable insights. Salary measures such as average and median salary are used to analyze compensation levels while accounting for extreme values. Demand-related measures, including total job postings, capture the size of the AI labor market, while remote share evaluates the prevalence of flexible work models. Additionally, an experience-based salary ratio compares executive and entry-level compensation to highlight salary disparities and the value placed on experience. Together, these dynamic measures support a comprehensive analysis of salaries, demand, remote work, and workforce structure across the global AI job market.

```
1 Average Salary = AVERAGE(cleaned_dataset[salary_usd])

1 Remote Share % = DIVIDE(CALCULATE([Total Postings], 'cleaned_dataset'[remote_ratio] = "Remote"), [Total Postings])

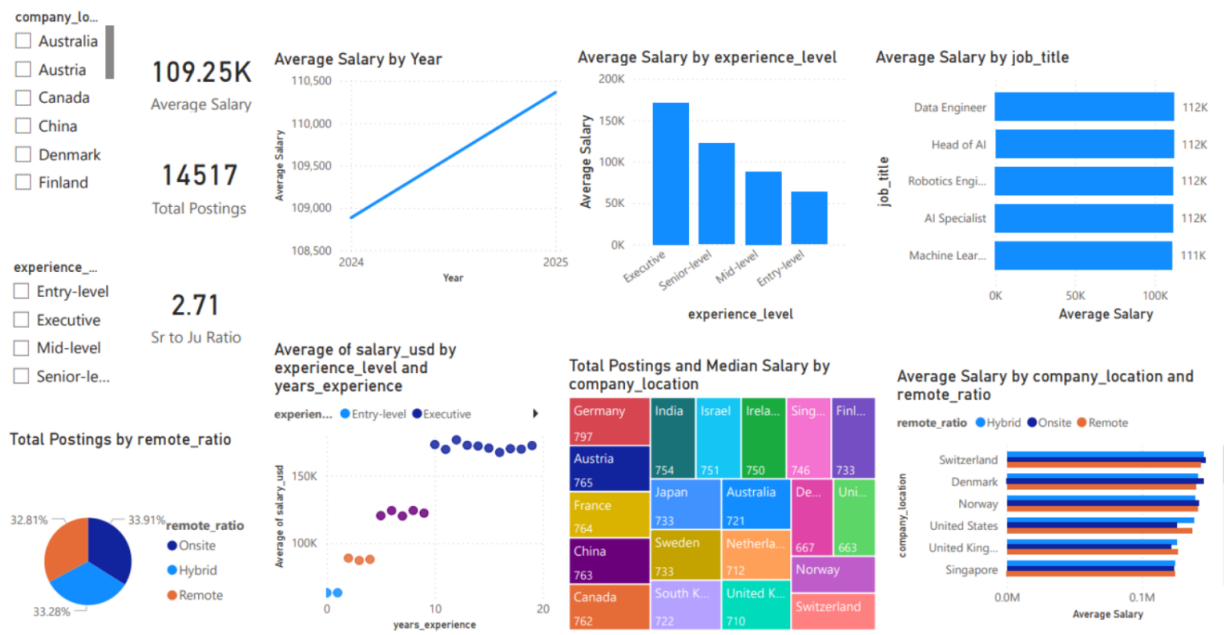
1 Sr to Ju Ratio = VAR SrSalary = CALCULATE(AVERAGE('cleaned_dataset'[salary_usd]), 'cleaned_dataset'[experience_level] = "Executive")
VAR JuSalary = CALCULATE(AVERAGE('cleaned_dataset'[salary_usd]), 'cleaned_dataset'[experience_level] = "Entry-level")
RETURN
DIVIDE(SrSalary, JuSalary, 0)

1 Median Salary = MEDIAN(cleaned_dataset[salary_usd])

1 Salary Diff = CALCULATE([Average Salary], 'cleaned_dataset'[remote_ratio] = "Remote") - CALCULATE([Average Salary], 'cleaned_dataset'[remote_ratio] = "Onsite")

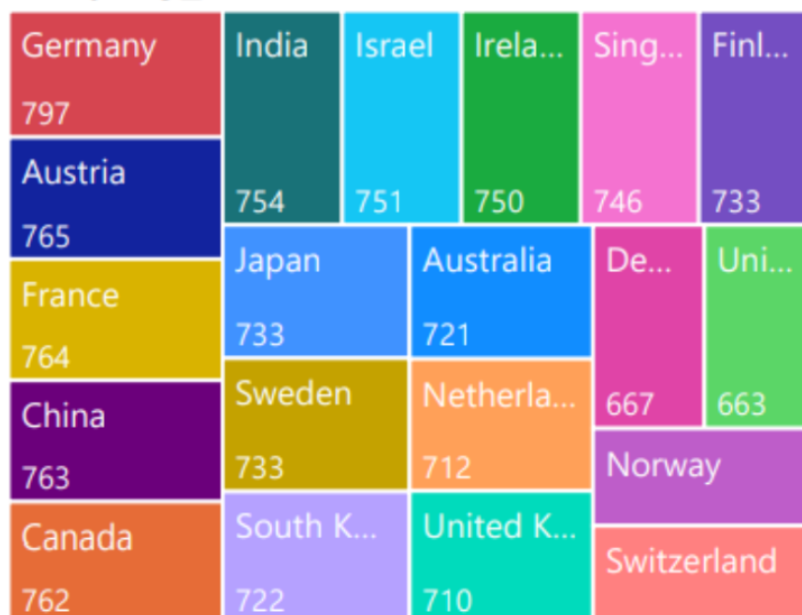
1 Total Postings = COUNTROWS('cleaned_dataset')
```

1.9 Data Visualization



1.9.1 Total Postings and Median Salary by Company Location

Total Postings and Median Salary by company_location



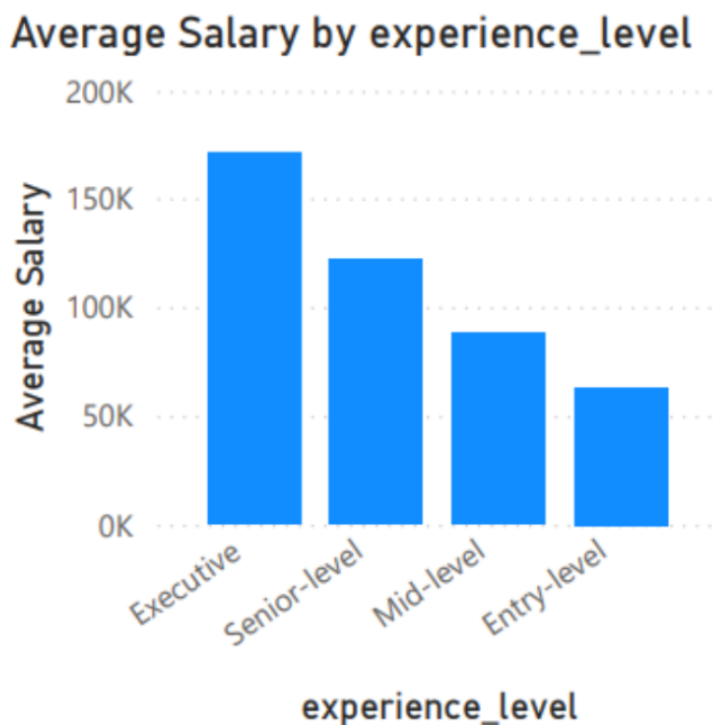
Interpretation

This chart compares AI job demand (number of postings) and median salary across different countries. Countries like Germany, Austria, France, China, Canada, India, Israel, Singapore, Finland, Japan, Sweden, South Korea, Australia, Netherlands, UK, Norway, and Switzerland appear at the top.

Insights

- Germany, Austria, and France show the highest number of postings, indicating strong AI hiring demand in Europe.
- Countries like Switzerland, Norway, and the UK, while sometimes having slightly fewer postings, are known for higher salary levels, suggesting quality-over-quantity markets.
- This highlights two types of AI markets:
 - High-volume hiring markets (Germany, France, India)
 - High-paying specialist markets (Switzerland, Norway)
- Consulting insight: Companies can choose countries based on cost vs talent availability trade-offs.

1.9.2 Average Salary by Experience Level



Interpretation

This bar chart shows how average salary increases with experience level:

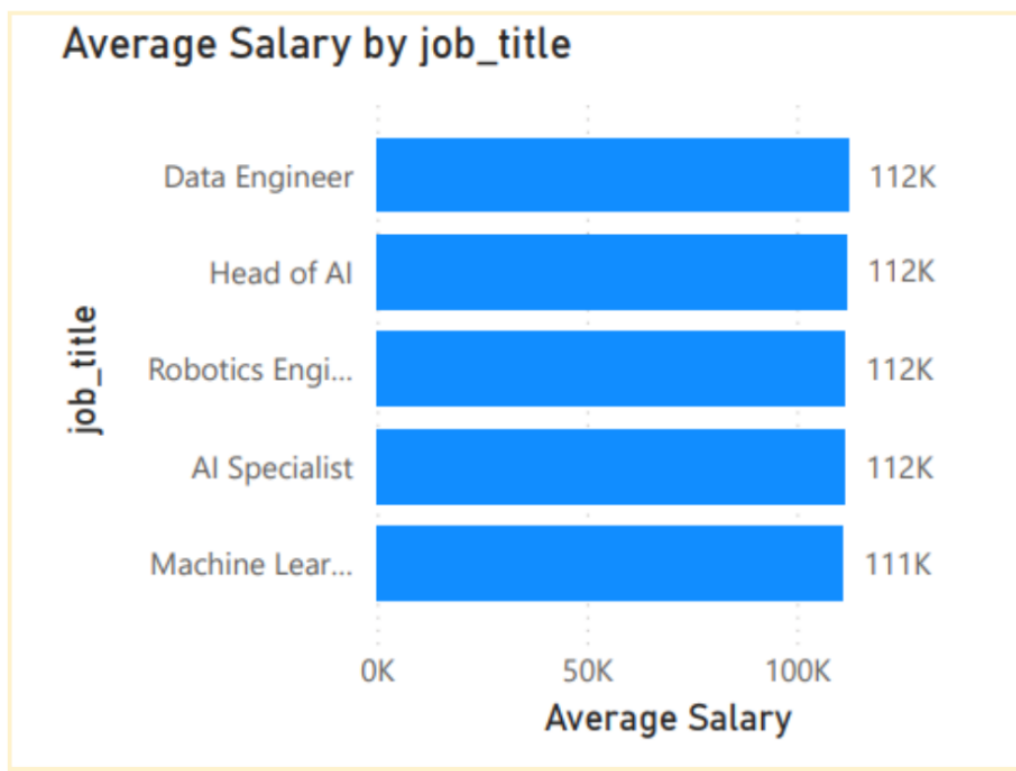
- Entry-level

- Mid-level
- Senior-level
- Executive

Insights

- Salaries increase sharply as experience increases.
- Executive-level roles earn the highest average salary, far above other levels.
- The largest salary jump occurs between Senior-level and Executive, reflecting leadership and strategic responsibility.
- Talent strategy insight: Retaining senior and executive AI talent is financially critical, as replacement costs are high.

1.9.3 Average Salary by Job Title



Interpretation

This chart compares average salaries across AI-related job titles such as:

- Data Engineer

- Head of AI
- Robotics Engineer
- AI Specialist
- Machine Learning Engineer

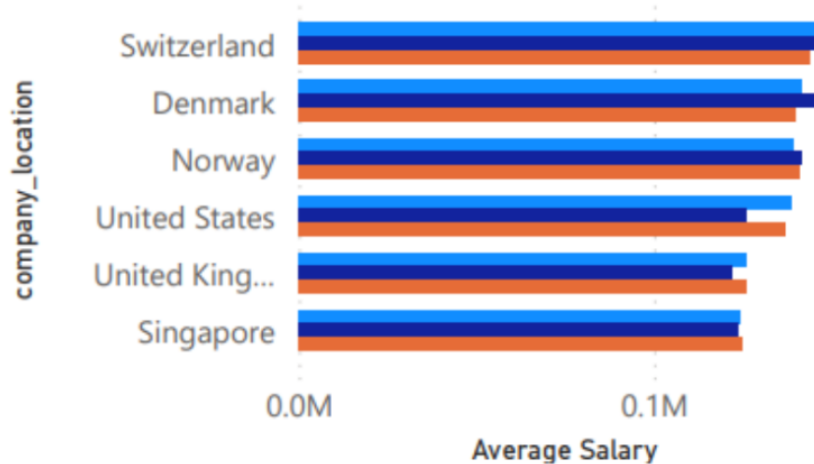
Insights

- Salaries are relatively close (around \$111K–\$112K), indicating market standardization across core AI roles.
- Head of AI ranks among the highest, reflecting leadership + technical expertise.
- Technical roles like Data Engineer and ML Engineer remain highly valued.
- Market insight: AI skills are priced consistently, showing mature demand rather than speculative hype.

1.9.4 Average Salary by Company Location and Remote Ratio

Average Salary by company_location and remote_ratio

remote_ratio ● Hybrid ● Onsite ● Remote



Interpretation

This graph compares average salary across countries by work type:

- Onsite

- Hybrid
- Remote

Insights

- Switzerland, Denmark, Norway, the US, UK, and Singapore offer the highest salaries, regardless of work mode.
- Remote roles do not always pay less; in some countries, remote salaries are competitive or higher.
- Hybrid work often sits between onsite and remote in terms of pay.
- Strategic insight: Remote work enables companies to access high-paying markets without full relocation costs.

1.9.5 KPI Cards (Top Metrics)



Interpretation

These KPIs summarize the dashboard:

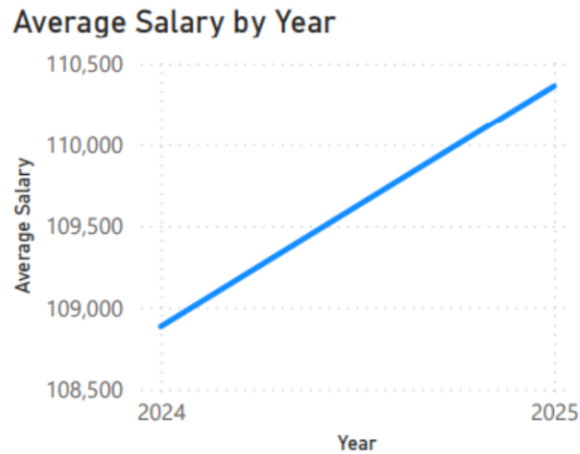
- Average Salary: \$109,250
- Total Postings: 14,517
- Senior to Junior Ratio: 2.71

Insights

- AI roles are high-paying globally.
- Demand is strong and scalable.
- The Senior-to-Junior ratio of 2.71 shows:
 - A market that strongly values experience

- Potential talent pipeline gaps at junior levels
- Workforce insight: Companies may face future shortages if junior talent is not developed.

1.9.6 Average Salary by Year (2024 vs 2025)



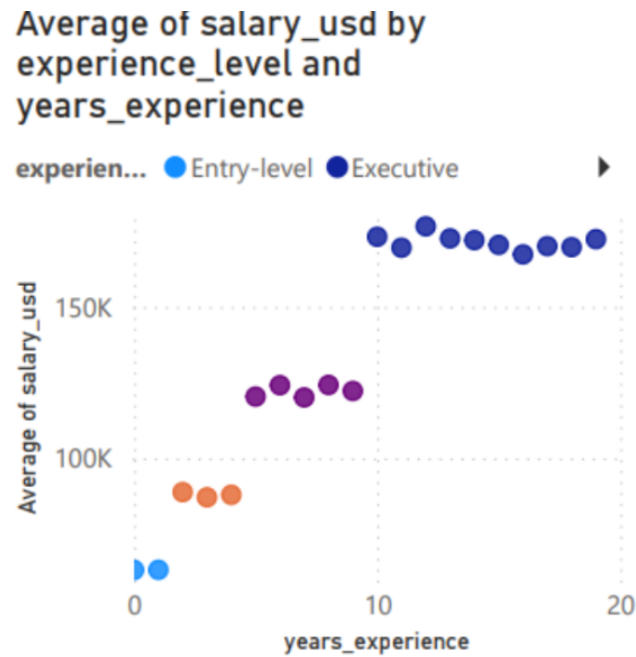
Interpretation

This line chart compares average AI salaries over time.

Insights

- Salaries show a slight upward trend from 2024 to 2025.
- Growth is steady, not explosive, indicating a stable and mature market.
- AI compensation is resilient, even in uncertain economic conditions.
- Business insight: AI roles are a long-term investment, not a short-term bubble.

1.9.7 Average Salary by Experience Level and Years of Experience



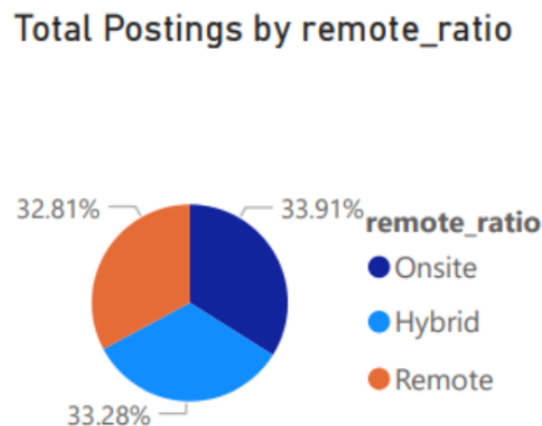
Interpretation

This chart shows how salary grows with years of experience, split by experience level.

Insights

- Salary increases consistently with experience.
- Entry-level salaries grow slowly at first.
- Executive salaries rise much faster, especially after approximately 10 years of experience.
- Career insight: Long-term experience in AI leads to exponential value creation, especially in leadership roles.

1.9.8 Total Postings by Remote Ratio



Interpretation

This pie chart shows job distribution:

- Onsite: 33.91%
- Hybrid: 33.28%
- Remote: 32.81%

Insights

- The market is almost perfectly balanced between onsite, hybrid, and remote work.
- No single work model dominates.
- This reflects post-pandemic normalization, not a remote-only world.
- HR insight: Companies must offer flexibility to stay competitive.

The global AI job market is strong, well-distributed, and mature. Salaries increase significantly with experience, leadership roles command a premium, and demand is geographically diverse. Remote and hybrid work have become standard rather than exceptional, enabling organizations to optimize cost, talent access, and flexibility.

1.10 Conclusion and Key Insights

This project provided a comprehensive analysis of the global AI job market, focusing on salary trends, job demand, experience levels, and remote work distribution. By collecting, cleaning, and structuring

the dataset into a well-organized data warehouse, we were able to create meaningful insights using SQL queries, ROLAP analysis, DAX measures, and interactive dashboards. Key findings include:

- **Salary Trends:** AI salaries are generally high worldwide, with executive and senior-level roles commanding a significant premium. Salaries increase steadily with experience and show a stable upward trend across years.
- **Job Demand and Locations:** High-volume hiring occurs in countries like Germany, France, and India, while high-paying specialist roles are concentrated in Switzerland, Norway, and the UK. Companies can balance cost and talent availability by leveraging these patterns.
- **Remote Work Patterns:** Onsite, hybrid, and remote work are nearly equally distributed, reflecting post-pandemic workforce normalization. Remote work offers access to competitive salaries without geographic constraints.
- **Talent Pipeline Insights:** The senior-to-junior salary ratio indicates a market that values experience highly, highlighting the importance of developing junior AI talent to prevent future talent shortages.

Challenges Faced

- **Data Quality:** Handling missing, inconsistent, or improperly formatted data required extensive cleaning and validation to ensure accuracy.
- **Outlier Management:** Extreme salary values and unusual data points had to be carefully identified and treated to avoid skewing insights.
- **Integration and Modeling:** Designing a scalable star schema for the data warehouse and connecting fact and dimension tables required careful planning to optimize query performance.
- **Analytical Complexity:** Extracting meaningful business insights from multi-dimensional data across countries, roles, and remote work arrangements demanded precise use of SQL, ROLAP, and DAX measures.

Overall, this project demonstrates the value of structured data and BI tools in guiding HR and recruitment decisions for the AI sector. It equips organizations with actionable insights to optimize compensation, identify talent hotspots, plan workforce needs, and leverage flexible work models efficiently.